MLIR: Scaling Compiler Infrastructure for Domain Specific Computation [1]

Federico Bruzzone,[1] PhD Candidate

Milan, Italy – 18 March 2026

[1] ADAPT Lab – University of Milan,
Website: federicobruzzone.github.io,
Github: github.com/FedericoBruzzone,
Email: federico.bruzzone@unimi.it
Slides: TODO

MLIR: Multi-Level Intermediate Representation

Part of the LLVM project, the MLIR is a novel approach to building **reusable**, **modular**, and **extensible** compiler infrastructure.

MLIR aims to address software fragmentation, improve compilation for heterogeneous hardware, significantly reduce the cost of building **domain specific compilers**, and aid in connecting existing compilers together.

Why another compiler infrastructure?

Although the *one size fits all* approach of traditional compilers (e.g., LLVM [2] and JVM [3]) has been successful for general-purpose programming, it has shown limitations in the context of domain-specific applications.

Many problems are better modeled at a **higher-** or **lower-level abstraction** — e.g., source-level static analysis of C++/Rust is difficult on LLVM IR.

Hence, many languages and frameworks developed their own intermediate representations (IRs) to leverage the **semantic information** of their domain — including TensorFlow's XLA HLO, PyTorch's Glow, Rust's MIR, Swift's SIL, Clang's CIL, and so on.

While domain-specific IRs are well-understood, their *high engineering costs* often lead to compromised infrastructure quality. This results in *suboptimal compilers* plagued by bugs, latency, and a poor debugging experience [1].

MLIR to the rescue

MLIR directly addresses these issues by making it **cheap** to design and **introduce** new abstraction layers.

It achieves this by:

- standardizing the Static Single Assignment (SSA)-based IR data structures,
- providing a declarative system for defining IR *dialects*, and
- providing a wide range of common infrastructure including documentation, parsing and printing logic, location tracking, multithreaded compilation support, pass management.

Design Principles

- **Parsimony**: Apply *Occam's razor* to builtin semantics, concepts, and programming interface. Specify invariants once, but verify correctness throughout $\Longrightarrow$ *extensibility*.
- **Traceability**: Retain rather than recover information. Declare rules and properties to enable transformation, rather than step wise imperative specification $\Longrightarrow$ *composability*.
- **Progressivity**: Premature lowering is the root of all evil. Beyond representation layers, allow multiple transformation paths that lower individual regions on demand $\Longrightarrow$ *reusability*.

Little Builtin, Everything Customizable *[Parsimony]*

- The system is based on a minimal number of fundamental concepts, leaving most of the intermediate representation fully **customizable**.
- A handful of abstractions—types, operations and attributes—should be used to express *everything else*, allowing fewer and more consistent abstractions that are easy to **comprehend**, **extend**, and **adopt**.
- A success criterion for customization is the possibility to express a diverse set of abstractions including **ML graphs**, ASTs, mathematical abstractions such as **polyhedral**, CFGs and instruction-level IRs such as **LLVM IR**, without hard-coding concepts.

SSA and Regions *[Parsimony]*

- **SSA** [4] makes dataflow analysis *simple* and *sparse*. However, while many existing IRs use this flat, linearized CFG, representing higher level abstractions push introducing **nested regions**[2] as a first-class citizen — e.g., structured control flow, concurrency constructs, and closures.
- The (LLVM) normalization/canonicalization process is sacrificed due to the presence of multiple ways to represent the same semantics.
- The frontend is responsible for choosing the level of abstraction for the IR.

[2] A region is a single-entry, multi-exit CFG that can be nested inside an operation. It is a generalization of the concept of basic blocks and allows for more flexible control flow representation.

The Canonical Loop Structure

Pre-header, *header*, *latch*, and *body* is a prototypical loop structure.

```llvm
; for (int i = 0; i < n; ++i) { ... }
entry:
  br label %header
header:
  %i = phi i32 [ 0, %entry ], [ %inc, %latch ]
  %cmp = icmp slt i32 %i, %n
  br i1 %cmp, label %body, label %multi-exit
latch:
  %inc = add i32 %i, 1
  br label %header
body:
  ; loop body
  br label %latch
multi-exit:
  ; code after the loop
```

```mlir
// A simple loop from 0 to 10 with a step of 1
scf.for %i = %c0 to %c10 step %c1 {
  // Loop body goes here
  // %i is the induction variable
  "some.operation"(%i) : (index) -> ()
}
```

```mlir
// An affine loop: optimized for polyhedral compilation
affine.for %i = 0 to 10 {
    %val = affine.load %buffer[%i] : memref<10xf32>
    // ... operations ...
}
```

Maintain Higher-Level Semantics *[Progressivity]*

- Attempts to **recover** abstract semantics once lowered are fragile and often **fail** to capture the full semantics.
- The system should maintain the structure of computations and **progressively lower** to the hardware abstraction.
- Removing structured control flow — i.e. lowering to a CFG — essentially means no further transformations will be performed that exploits the structure.
- Previous compilers have been introducing multiple fixed levels of abstraction in their pipeline causing **phase ordering** issues.

Declaration and Validation *[Parsimony|Traceability]*

- Defining representation modifiers should be as simple as introducing new abstractions.
- Common transformations should be implementable as **rewrite rules** expressed declaratively.
- Although rewriting systems are well-studied, the MLIR's extensibility opens up new challenges.
- While verification, testing, and translation validation [5] are useful a more robust approach to combining all these techniques for **extensible** and **modular** IRs.

Source Location Tracking *[Traceability]*

- **Lack-of-transparency** in complex compilation systems is ubiquitous. This is particularly problematic when compiling safety-critical and sensitive applications (cf. `WYSINWYX` by Balakrishnan et al. [6]).
- Thus, the **provenance** of an operation — including its original location and applied transformations — should be easily traceable within MLIR.
- One indirect goal of accurately propagating high-level information to the lower levels is to help support **secure** and **traceable** compilation.

Intermediate Representation Design

MLIR has a *generic* textual representation that supports MLIR's extensibility and fully reflects the in-memory representation, which is paramount for **traceability**, manual IR **validation** and **testing**. Extensibility comes with the burden of verbosity, which can be compensated by the custom syntax that MLIR supports.

```mlir
// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = ()[s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
  // Block are lists of operations.
  "affine.for"(%arg0) ({
  ^bb0(%arg5: index):
    // Ops use and define typed values, which obey SSA.
    %0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %2 = "std.mulf"(%0, %1) : (f32, f32) -> f32
    %3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1} : (memref<?xf32>, index, index) -> f32
    %4 = "std.addf"(%3, %2) : (f32, f32) -> f32
    "affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1} : (f32, memref<?xf32>, index, index) -> ()
    // Blocks end with a terminator Op.
    "affine.terminator"() : () -> ()
  // Ops have a list of attributes.
  }) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
  "affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
```

IR: *Operations*

The unit of semantics in MLIR is an **operation** (Op): *instruction*, *function* and *module* are modeled as Ops.

MLIR does not have a fixed set of Ops, but allows user-defined extensions.

The infrastructure provides a declarative syntax for defining Ops based on the well-known **TableGen**.[3]

tablegen

```
// An Op is a TableGen definition that inherits the "Op" class parameterized
with the Op name
def LeakyReluOp: Op<"leaky_relu",
    // and a list of traits used for verification and optimization.
    [NoSideEffect, SameOperandsAndResultType]> {
  // The body of the definition contains named fields for a one-line
  // documentation summary for the Op.
  let summary = "Leaky Relu operator";
  // The Op can also a full-text description that can be used to generate
  // documentation for the dialect.
  let description = [{
    Element-wise Leaky ReLU operator x -> x >= 0 ? x : (alpha * x) }];
  // Op can have a list of named arguments, which include typed operands and attributes.
  let arguments = (ins AnyTensor:$input, F32Attr:$alpha);
  // And a list of named and typed outputs.
  let results = (outs AnyTensor:$output);
}
```

[3] https://llvm.org/docs/TableGen/

IR: *Operations* (cont.)

Ops have a **unique** opcode: the operation and its dialect.

Ops take and produce zero or more SSA *operands* and *results*.

Values represent runtime data and are fully typed to ensure compile-time knowledge.

Ops may also have *Attributes*, *Regions*, *Successor Blocks*, and *Laocation* Information.

```mlir
// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = ()[s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
  // Block are lists of operations.
  "affine.for"(%arg0) ({
  ^bb0(%arg5: index):
    // Ops use and define typed values, which obey SSA.
    %0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %2 = "std.mulf"(%0, %1) : (f32, f32) -> f32
    %3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1} : (memref<?xf32>, index, index) -> f32
    %4 = "std.addf"(%3, %2) : (f32, f32) -> f32
    "affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1} : (f32, memref<?xf32>, index, index) -> ()
    // Blocks end with a terminator Op.
    "affine.terminator"() : () -> ()
  // Ops have a list of attributes.
  }) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
  "affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
```

IR: *Attributes*

MLIR **attributes** contain compile-time information about Ops.

Attributes are typed (e.g., integer, string), and each Op instance has an open key-value dictionary from string names to attribute values.

Attributes derive their meaning either from the **Op semantics** or from the **dialect** they are associated with.

As with opcodes, there is no fixed set of attributes

```mlir
// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = ()[s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
  // Block are lists of operations.
  "affine.for"(%arg0) ({
  ^bb0(%arg5: index):
    // Ops use and define typed values, which obey SSA.
    %0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32
    %2 = "std.mulf"(%0, %a1) : (f32, f32) -> f32
    %3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1} : (memref<?xf32>, index, index) -> f32
    %4 = "std.addf"(%3, %2) : (f32, f32) -> f32
    "affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1} : (f32, memref<?xf32>, index, index) -> ()
    // Blocks end with a terminator Op.
    "affine.terminator"() : () -> ()
  // Ops have a list of attributes.
  }) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
  "affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> ()
```

IR: *Location Information*

MLIR provides a compact representation for **location information**, and encourages the processing and propagation of this information throughout the system, following the **traceability** principle.

It can be used to keep the source program stack trace that produced an Op, to generate debug information.

```mlir
// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = ()[s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
  // Block are lists of operations.
  "affine.for"(%arg0) ({
  ^bb0(%arg5: index):
    // Ops use and define typed values, which obey SSA.
    %0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32 loc("kernel.c":10:12)
    %1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)} : (memref<?xf32>, index) -> f32 loc("kernel.c":11:12)
    %2 = "std.mulf"(%0, %a1) : (f32, f32) -> f32 loc(fused["kernel.c":12:15, "params.h":5:2])
    %3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1} : (memref<?xf32>, index, index) -> f32 loc("kernel.c":13:8)
    %4 = "std.addf"(%3, %2) : (f32, f32) -> f32 loc("kernel.c":13:14)
    "affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1} : (f32, memref<?xf32>, index, index) -> () loc("kernel.c":13:5)
    // Blocks end with a terminator Op.
    "affine.terminator"() : () -> ()
  // Ops have a list of attributes.
  }) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> () loc("kernel.c":9:5)
  "affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3} : (index) -> () loc("kernel.c":8:3)
```

Thank You!

Bibliography

[1] C. Lattner *et al.*, "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14. doi: 10.1109/CGO51591.2021.9370308.

[2] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization, 2004. CGO 2004.*, 2004, pp. 75–86.

[3] T. Lindholm, F. Yellin, G. Bracha, and A. Buckley, *The Java virtual machine specification*. Addison-wesley, 2013.

[4] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, Oct. 1991, doi: 10.1145/115372.115320.

[5] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Tools and Algorithms for the Construction and Analysis of Systems*, B. Steffen, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 151–166.

[6] G. Balakrishnan and T. Reps, "WYSINWYX: What you see is not what you eXecute," *ACM Trans. Program. Lang. Syst.*, vol. 32, no. 6, Aug. 2010, doi: 10.1145/1749608.1749612.